

Rapport Technique : Migration de l'Authentification

Projet : IncidentFlow V2 Sujet : Transition d'un environnement Dev-Mock vers Keycloak IAM

1. Introduction et Contexte

Dans le cadre de l'évolution de la plateforme IncidentFlow vers sa version 2, l'une des exigences majeures de sécurité consistait à remplacer le système d'authentification simulé (`dev-mock`) par un véritable système de Gestion des Identités et des Accès (IAM). Le choix s'est porté sur **Keycloak**, une solution Open Source robuste qui permet de mettre en place une architecture Zero-Trust, d'assurer une authentification unique (SSO) et de gérer les rôles de manière centralisée et sécurisée via les standards OAuth2 et OpenID Connect (OIDC).

2. Implémentation et Architecture

La transformation a requis une refonte complète de la chaîne de sécurité de bout en bout :

2.1. Frontend (React / Vite)

- **Remplacement de l'UI** : L'ancienne page de connexion factice (Mock Login) a été supprimée au profit du portail captif standardisé de Keycloak.
- **Intégration Keycloak-JS** : La librairie officielle a été greffée directement au point d'entrée (`main.jsx`) avec l'option `onLoad: 'login-required'` , empêchant le rendu de l'application React tant qu'un jeton valide n'a pas été négocié.
- **Gestion du Cycle de Vie** : Mise en place d'un mécanisme de rafraîchissement automatique du jeton (Auto-refresh token) en arrière-plan et injection systématique de ce jeton sous forme de *Bearer Token* dans les en-têtes HTTP de chaque requête sortante.

2.2. Backend (Spring Boot 3)

- **Basculement de Profil** : Abandon du profil `dev-mock` (qui autorisait toutes les requêtes) au profit du profil `prod` .
- **OAuth2 Resource Server** : Configuration de Spring Security en tant que *Resource Server*. Les requêtes ne sont plus validées par l'en-tête factice `X-Mock-User` , mais par la vérification cryptographique (JWKS) d'un jeton JWT signé par Keycloak.

2.3. Infrastructure (Docker Compose)

- Le serveur Keycloak a été conteneurisé aux côtés de la base de données PostgreSQL avec l'importation automatique d'un *Realm* (domaine d'authentification) prédéfini, contenant les clients et les utilisateurs de base (admin, operateur).

3. Défis Techniques et Résolutions (Troubleshooting)

L'intégration de Keycloak dans un environnement micro-services Docker complexe a soulevé plusieurs défis réseau et de configuration qui ont été méthodiquement résolus :

Problème 1 : Connexion Refusée (ERR_CONNECTION_REFUSED)

- **Symptôme** : Le navigateur affichait une erreur indiquant que la page Keycloak était inaccessible au lancement.
- **Analyse** : Les configurations initiales pointaient vers le port `8180` par défaut, mais le fichier `docker-compose.yml` exposait explicitement Keycloak sur le port `8181` .

- **Résolution** : Harmonisation des ports ciblés dans le fichier `keycloak.js` (React) et `application-prod.properties` (Spring Boot).

Problème 2 : Erreur Keycloak "We are sorry... Page not found"

- **Symptôme** : L'URL de redirection générée par `keycloak.js` aboutissait sur une erreur 404 native de Keycloak.
- **Analyse** : L'adaptateur JavaScript requêtait le realm nommé `incidentflow`. Cependant, l'inspection du fichier JSON d'importation Keycloak (`realm-export.json`) a révélé que le domaine s'appelait en réalité `incidentflow-realm`.
- **Résolution** : Ajustement de la configuration pour cibler le nom de domaine exact attendu par le serveur IAM.

Problème 3 : Refus API en cascade (Erreur 401 Unauthorized - Frontend)

- **Symptôme** : Bien qu'authentifié, l'utilisateur voyait son tableau de bord vide car toutes les requêtes API (workflows, incidents, utilisateurs) revenaient en 401.
- **Analyse** : L'architecture de `App.jsx` comportait une dette technique : elle possédait son propre état interne d'authentification et sa propre fonction `getHeaders()`. Elle contournait la nouvelle configuration Keycloak et envoyait un jeton vide aux API.
- **Résolution** : Nettoyage complet de la logique d'état dans `App.jsx` pour la lier dynamiquement à l'état de `window.keycloak`, garantissant que chaque `fetch` attache le jeton JWT réel.

Problème 4 : Le Piège du Réseau Docker (Signature JWT Invalide)

- **Symptôme** : L'erreur 401 persistait, et les logs du Backend (Spring Boot) affichaient : `Connection refused` lors du fetch des certificats `certs`.
- **Analyse** : Pour vérifier un jeton, Spring Boot doit télécharger les clés publiques de Keycloak. Il tentait de le faire via `http://localhost:8181`. Or, dans le contexte d'un conteneur Docker, `localhost` désigne le conteneur Spring Boot lui-même (où aucun Keycloak ne tourne).
- **Résolution** : Découplage de la configuration JWT dans Spring Boot :
 - `issuer-uri` : Fixé sur `http://localhost:8181/...` (pour valider la chaîne "iss" présente dans le jeton émis par le navigateur).
 - `jwk-set-uri` : Fixé sur `http://incidentflow-keycloak-v2:8080/...` (pour forcer Spring Boot à utiliser le DNS interne de Docker afin de télécharger les clés publiques de manière invisible pour l'utilisateur).

Problème 5 : Écrasement de Configuration (Priorité des profils)

- **Symptôme** : Les corrections appliquées au backend (Problème 4) ne semblaient pas être prises en compte par Spring Boot.
- **Analyse** : Les modifications étaient injectées dans `application.properties`, mais le profil actif (`SPRING_PROFILES_ACTIVE=prod`) forçait Spring Boot à écraser ces variables avec un fichier spécifique `application-prod.properties` qui contenait toujours les anciennes valeurs erronées.
- **Résolution** : Modification explicite du fichier de production pour propager définitivement l'architecture Docker interne, suivi d'un `docker compose up --build`.

Problème 6 : Déconnexion Brutale (ERR_CONNECTION_RESET)

- **Symptôme** : Le navigateur du développeur a renvoyé des erreurs `ERR_CONNECTION_RESET` et `ERR_SOCKET_NOT_CONNECTED`.
- **Analyse** : Il s'agissait d'un simple conflit de timing. Des requêtes API ont été déclenchées au moment exact où le conteneur Spring Boot redémarrait pour appliquer les nouvelles configurations de

production. Le redémarrage complet de l'application (prenant environ 14 secondes) a interrompu les sockets TCP existantes.

- **Résolution** : Attente de la stabilisation du cycle de vie du conteneur et rafraîchissement applicatif, entraînant une réussite totale à 100%.

Problème 7 : Désynchronisation des Utilisateurs (Erreur "User not found")

- **Symptôme** : Lors de la création ou de la modification d'un incident, l'API renvoyait l'erreur `User not found for email: admin@netmar.com`.
- **Analyse** : Bien que Keycloak authentifie parfaitement l'utilisateur, le backend Spring Boot cherche à lier chaque incident à une entité "Utilisateur" dans sa propre base PostgreSQL locale (pour préserver l'intégrité relationnelle de la base de données métiers). L'utilisateur `admin@netmar.com`, présent dans Keycloak, n'avait pas été provisionné dans le script d'initialisation de la base (`DataInitializer.java`) qui ne contenait que d'anciens comptes fictifs.
- **Résolution** : Mise à jour de la logique d'amorçage de Spring Boot pour synchroniser et insérer dynamiquement les profils de base de Keycloak (`admin@netmar.com` et `operateur@netmar.com`) directement dans PostgreSQL au démarrage du serveur.

4. Conclusion

La migration de l'environnement factice vers Keycloak est un succès architectural total. Elle élève le standard de sécurité de l'application IncidentFlow au niveau des exigences de l'industrie (Enterprise-grade). Les différents obstacles rencontrés, symptomatiques de la complexité des micro-services (DNS asymétriques Docker, gestion asynchrone des états React, cycle de vie OIDC, intégrité référentielle des utilisateurs), ont été identifiés avec précision et résolus, dotant l'infrastructure d'une résilience et d'une sécurité optimales.